Share Cart via Link - Implementation Plan

Overview

Allow users to share their cart with others via a shareable link. Recipients can view the shared cart and optionally add items to their own cart.

Use Cases

1. **User shares cart with family/friends** - "Check out what I'm buying!"
2. **Shopping list sharing** - Share grocery list with household members
3. **Gift suggestions** - Share cart as a wishlist
4. **Collaborative shopping** - Multiple people can view and add same items

Architecture Design

Option 1: Token-Based Sharing (Recommended) ✅

Pros:

- Secure and private
- Can set expiration
- Can track who accessed
- Can revoke access
- Lightweight (no data duplication)

Cons:

- Requires token management
- Slightly more complex

Option 2: Snapshot-Based Sharing

Pros:

- Simple implementation
- Cart snapshot preserved even if original changes

Cons:

- Data duplication
- No real-time updates
- More storage required

Recommended Approach: Token-Based with Snapshot

Hybrid approach combining both:

1. Generate unique shareable token
2. Create snapshot of cart at time of sharing
3. Token links to snapshot (not live cart)
4. Snapshot is read-only
5. Recipients can copy items to their cart

Database Schema

New Table: TM_SharedCart

```
CREATE TABLE "TM_SharedCart" (
    "Id" SERIAL PRIMARY KEY,
    "ShareToken" VARCHAR(100) UNIQUE NOT NULL,  -- Unique shareable token (GUID)
    "UserId" INTEGER NULL,                       -- Original cart owner (if logged in)
    "GuestId" VARCHAR(100) NULL,                 -- Original cart owner (if guest)
    "StoreCode" VARCHAR(50) NOT NULL,
    "CompanyCode" VARCHAR(50) NULL,
    "Title" VARCHAR(200) NULL,                   -- Optional: "My Grocery List"
    "Description" TEXT NULL,                     -- Optional description
    "TotalItems" INTEGER NOT NULL,
    "TotalAmount" DECIMAL(18,2) NOT NULL,
    "ExpiresAt" TIMESTAMP NULL,                  -- Optional expiration
    "IsActive" BOOLEAN NOT NULL DEFAULT true,
    "IsPublic" BOOLEAN NOT NULL DEFAULT false,   -- Public vs private link
    "ViewCount" INTEGER NOT NULL DEFAULT 0,
    "CopyCount" INTEGER NOT NULL DEFAULT 0,      -- How many times copied
    "CreatedDate" TIMESTAMP NOT NULL,
    "UpdatedDate" TIMESTAMP NOT NULL
);

CREATE INDEX "IX_SharedCart_ShareToken" ON "TM_SharedCart"("ShareToken");
CREATE INDEX "IX_SharedCart_UserId" ON "TM_SharedCart"("UserId");
CREATE INDEX "IX_SharedCart_ExpiresAt" ON "TM_SharedCart"("ExpiresAt");
```

New Table: TM_SharedCartItem

```
CREATE TABLE "TM_SharedCartItem" (
    "Id" SERIAL PRIMARY KEY,
    "SharedCartId" INTEGER NOT NULL,
    "ArticleNumber" VARCHAR(50) NOT NULL,
    "VariantId" VARCHAR(50) NULL,
    "ProductNameEn" VARCHAR(500) NULL,
    "ProductNameAr" VARCHAR(500) NULL,
    "ProductImage" VARCHAR(500) NULL,
    "BrandId" INTEGER NULL,
    "BrandName" VARCHAR(200) NULL,
    "CategoryId" VARCHAR(100) NULL,
    "CategoryName" VARCHAR(200) NULL,
    "Quantity" INTEGER NOT NULL,
    "UOM" VARCHAR(50) NULL,
    "VariantWeight" VARCHAR(50) NULL,
    "Price" DECIMAL(18,2) NOT NULL,
    "Discount" DECIMAL(18,2) NOT NULL DEFAULT 0,
    "Tax" DECIMAL(18,2) NOT NULL DEFAULT 0,
    "TotalAmount" DECIMAL(18,2) NOT NULL,
    "IsAvailable" BOOLEAN NOT NULL DEFAULT true, -- Product still available?
    "CreatedDate" TIMESTAMP NOT NULL,

    FOREIGN KEY ("SharedCartId") REFERENCES "TM_SharedCart"("Id") ON DELETE CASCADE
);

CREATE INDEX "IX_SharedCartItem_SharedCartId" ON "TM_SharedCartItem"("SharedCartId");
CREATE INDEX "IX_SharedCartItem_ArticleNumber" ON "TM_SharedCartItem"("ArticleNumber");
```

Optional: TM_SharedCartAccess (For Analytics)

```
CREATE TABLE "TM_SharedCartAccess" (
    "Id" SERIAL PRIMARY KEY,
    "SharedCartId" INTEGER NOT NULL,
    "AccessedByUserId" INTEGER NULL,
    "AccessedByGuestId" VARCHAR(100) NULL,
    "IPAddress" VARCHAR(50) NULL,
    "UserAgent" TEXT NULL,
    "Action" VARCHAR(50) NOT NULL,  -- 'View', 'Copy', 'AddToCart'
    "AccessedDate" TIMESTAMP NOT NULL,

    FOREIGN KEY ("SharedCartId") REFERENCES "TM_SharedCart"("Id") ON DELETE CASCADE
);

CREATE INDEX "IX_SharedCartAccess_SharedCartId" ON "TM_SharedCartAccess"("SharedCartId");
```

API Endpoints

1. Share Cart (Create Shareable Link)

POST `/api/web/cart/share`

Request:

```
{
  "userId": 123,           // or null for guest
  "guestId": "guest-uuid", // or null for logged-in user
  "storeCode": "STORE001",
  "title": "My Weekly Groceries",
  "description": "Items for this week",
  "expiresInDays": 7,      // Optional: 0 = no expiration
  "isPublic": false        // Private link by default
}
```

Response:

```
{
  "status": 1,
  "message": "Cart shared successfully",
  "data": {
    "shareToken": "abc123xyz789",
    "shareUrl": "https://app.com/shared-cart/abc123xyz789",
    "expiresAt": "2024-04-01T00:00:00Z",
    "totalItems": 15,
    "totalAmount": 250.50
  }
}
```

Logic:

1. Validate user has items in cart
2. Generate unique share token (GUID)
3. Create snapshot of current cart items
4. Save to TM_SharedCart and TM_SharedCartItem
5. Return shareable URL

---

2. Get Shared Cart Details

GET `/api/web/cart/shared/{shareToken}`

Response:

```
{
  "status": 1,
  "message": "Shared cart retrieved successfully",
  "data": {
    "shareToken": "abc123xyz789",
    "title": "My Weekly Groceries",
    "description": "Items for this week",
    "storeCode": "STORE001",
    "totalItems": 15,
    "totalAmount": 250.50,
    "createdDate": "2024-03-25T10:00:00Z",
    "expiresAt": "2024-04-01T00:00:00Z",
    "isExpired": false,
    "items": [
      {
        "id": 1,
        "articleNumber": "ART001",
        "productNameEn": "Fresh Milk",
        "productNameAr": "حليب طازج",
        "productImage": "/images/milk.jpg",
        "brandName": "Almarai",
        "quantity": 2,
        "uom": "Liter",
        "price": 15.50,
        "discount": 2.00,
        "totalAmount": 29.00,
        "isAvailable": true
      }
    ]
  }
}
```


Logic:

1. Validate share token exists and not expired
2. Increment view count
3. Log access (optional)
4. Check product availability in real-time
5. Return cart details

---

3. Copy Shared Cart to My Cart

POST `/api/web/cart/shared/{shareToken}/copy`

Request:

```
{
  "userId": 456,           // or null for guest
  "guestId": "guest-uuid", // or null for logged-in user
  "storeCode": "STORE001",
  "itemIds": [1, 2, 3],    // Optional: specific items, or all if null
  "replaceExisting": false // true = clear current cart first
}
```

Response:

```
{
  "status": 1,
  "message": "Items added to your cart successfully",
  "data": {
    "itemsAdded": 12,
    "itemsUnavailable": 3,
    "unavailableItems": [
      {
        "articleNumber": "ART005",
        "productNameEn": "Product Name",
        "reason": "Out of stock"
      }
    ],
    "cartTotal": 320.75
  }
}
```

Logic:

1. Validate share token
2. Check product availability and stock
3. Validate store match (optional)
4. Add available items to user's cart
5. Return summary with unavailable items

---

4. Get My Shared Carts (User's shared links)

GET `/api/web/cart/my-shares`

Query Params:

- `userId` or `guestId`
- `page` (default: 1)
- `pageSize` (default: 10)

Response:

```
{
  "status": 1,
  "message": "Shared carts retrieved successfully",
  "data": {
    "totalCount": 5,
    "page": 1,
    "pageSize": 10,
    "items": [
      {
        "shareToken": "abc123",
        "title": "My Weekly Groceries",
        "totalItems": 15,
        "totalAmount": 250.50,
        "viewCount": 12,
        "copyCount": 3,
        "createdDate": "2024-03-25T10:00:00Z",
        "expiresAt": "2024-04-01T00:00:00Z",
        "isExpired": false,
        "isActive": true
      }
    ]
  }
}
```

---

5. Delete/Revoke Shared Cart

DELETE `/api/web/cart/shared/{shareToken}`

Request:

```
{
  "userId": 123
}
```

Response:

```
{
  "status": 1,
  "message": "Shared cart deleted successfully"
}
```

---

6. Update Shared Cart (Optional)

PUT `/api/web/cart/shared/{shareToken}`

Request:

```
{
  "userId": 123,
  "title": "Updated Title",
  "description": "Updated description",
  "isActive": true
}
```

Implementation Steps

Phase 1: Database Setup

1. Create entity models (TM_SharedCart, TM_SharedCartItem, TM_SharedCartAccess)
2. Add DbSets to DB_Context
3. Create migration
4. Apply migration

Phase 2: Core Logic (DAL/BAL)

1. Create SharedCartDAL with methods:
    - `CreateSharedCartAsync()`
    - `GetSharedCartByTokenAsync()`
    - `CopySharedCartToUserCartAsync()`
    - `GetUserSharedCartsAsync()`
    - `DeleteSharedCartAsync()`
    - `UpdateSharedCartAsync()`
    - `IncrementViewCountAsync()`
    - `LogAccessAsync()`
2. Create SharedCartBAL (business logic layer)
3. Create ISharedCartRepository and ISharedCartServices interfaces

Phase 3: API Controllers

1. Create SharedCartController in WebMobileApi
2. Implement all endpoints
3. Add validation and error handling

Phase 4: Utilities

1. Token generator (GUID-based)
2. URL builder helper
3. Product availability checker
4. Expiration validator

Phase 5: Testing

1. Unit tests for DAL methods
2. Integration tests for API endpoints
3. Test expiration logic
4. Test concurrent access

Security Considerations

1. **Token Security:**
    - Use cryptographically secure random tokens (GUID)
    - Tokens should be long enough to prevent guessing
    - Consider adding rate limiting
2. **Expiration:**
    - Default expiration: 7 days
    - Cleanup expired shares periodically (background job)
3. **Privacy:**
    - Private links by default
    - Option to make public (for social sharing)
    - No personal information in shared cart
4. **Validation:**
    - Verify user owns cart before sharing
    - Validate store codes match when copying
    - Check product availability before adding

Performance Optimization

1. **Caching:**
    - Cache shared cart details (5-10 minutes)
    - Cache product availability status
2. **Indexing:**
    - Index on ShareToken (most common lookup)
    - Index on UserId for "my shares" query
    - Index on ExpiresAt for cleanup jobs
3. **Pagination:**
    - Paginate "my shares" list
    - Limit items per shared cart (e.g., max 100 items)
4. **Async Operations:**
    - All database operations should be async
    - Background job for expired cart cleanup

Frontend Integration

Share Button Flow:

1. User clicks "Share Cart" button
2. API call to create share
3. Show modal with:
    - Shareable link
    - Copy to clipboard button
    - QR code (optional)
    - Share via WhatsApp/Email buttons

View Shared Cart Flow:

1. User opens shared link
2. API call to get cart details
3. Show cart items with "Add to My Cart" button
4. Option to add all or select specific items

Additional Features (Optional)

1. **QR Code Generation** - For easy mobile sharing
2. **Social Sharing** - WhatsApp, Email, SMS integration
3. **Collaborative Carts** - Multiple people can edit same cart
4. **Cart Templates** - Save frequently used carts
5. **Analytics Dashboard** - Track share performance
6. **Notifications** - Notify when someone views/copies your cart

Estimated Effort

- **Database Setup:** 2 hours
- **DAL Implementation:** 4 hours

- **BAL Implementation:** 2 hours
- **API Controllers:** 3 hours
- **Testing:** 3 hours
- **Documentation:** 1 hour

Total: ~15 hours

Files to Create/Modify

New Files:

1. `TM_Ecommerce.DAL/Data/Order/TM_SharedCart.cs`
2. `TM_Ecommerce.DAL/Data/Order/TM_SharedCartItem.cs`
3. `TM_Ecommerce.DAL/Data/Order/TM_SharedCartAccess.cs`
4. `TM_Ecommerce.Model/Order/SharedCartDTO.cs`
5. `TM_Ecommerce.Repository/Order/ISharedCartRepository.cs`
6. `TM_Ecommerce.Services/Order/ISharedCartServices.cs`
7. `TM_Ecommerce.DAL/Order/SharedCartDAL.cs`
8. `TM_Ecommerce.BAL/Order/SharedCartBAL.cs`
9. `TM_ECommerce.WebMobileApi/Controllers/Web/SharedCartController.cs`

Modified Files:

1. `TM_Ecommerce.DAL/DB_Context.cs` - Add DbSets
2. `TM_Ecommerce/Program.cs` - Register services

Next Steps

1. Review and approve this plan
2. Confirm any modifications needed
3. Start implementation with Phase 1 (Database Setup)

Would you like me to proceed with the implementation? q w

ResinAura API - Setup Instructions

Prerequisites

- .NET 8.0 SDK
- MySQL Server installed and running

Setup Steps

1. Update Database Connection String

Edit `ResinAura_API/appsettings.json` and update the connection string with your MySQL credentials:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=localhost;Database=resinAuraDb;User=root;Password=YOUR_PASSWORD;"
}
```

2. Restore NuGet Packages

```
dotnet restore
```

3. Create Database Migration

```
cd ResinAura_API
dotnet ef migrations add InitialCreate
```

4. Apply Migration to Database

```
dotnet ef database update
```

5. Run the Application

```
dotnet run
```

API Endpoints

Register a New User

POST `/api/auth/register`

Request Body:

```
{
  "username": "testuser",
  "email": "test@example.com",
  "password": "password123"
}
```

Login

POST `/api/auth/login`

Request Body:

```
{
  "username": "testuser",
  "password": "password123"
}
```

Database Schema

The `Users` table will be created with the following structure:

- `Id` (int, primary key, auto-increment)
- `Username` (varchar, unique)
- `Email` (varchar, unique)
- `PasswordHash` (varchar)
- `CreatedAt` (datetime)

Notes

- Passwords are hashed using BCrypt
- No JWT tokens - simple username/password validation
- Unique constraints on username and email